

RELATÓRIO DE DESENVOLVIMENTO DO MVP: PIPELINE DE DADOS NA NUVEM

Análise de Pedidos da Olist Store (2016–2018)

Nome: Edson Antonio de Oliveira

GITHUB: <https://github.com/edsoliveira72>

Repositório MVP: https://github.com/edsoliveira72/MVP_PUC_Engenharia_Dados

Sprint: Engenharia de Dados – PUC RIO

SUMÁRIO

1. Contexto de Negócios e Formulação do Problema	3
1.1. Contexto de Negócios	3
1.2. Perguntas de Negócio	3
1.2.1. Domínio de Finanças e Meios de Pagamento	3
1.2.2. Domínio de Logística e Experiência do Cliente	3
1.2.3. Domínio de Desempenho Comercial e Cobertura de Vendedores	4
1.3. Especificação do Conjunto de Dados Brutos e Licenciamento	4
2. Ingestão e Carga dos Dados Brutos	5
2.1. Estratégia de Ingestão	5
2.2. Procedimento Operacional da Carga	5
3. Arquitetura do Pipeline de Dados e Processamento Bronze	5
3.1. Organização do Pipeline	5
3.2. Camada Bronze (01_ingestao_bronze)	6
4. Processamento da Camada Silver e Modelagem	7
4.1. Transformações e Qualidade na Camada Silver (02_ingestao_silver)	7
4.2. Modelagem Dimensional (Star Schema) e Catálogo de Dados	8
4.2.1. Esquema Estrela: Tabela gold_vendas_detalhadas	10
4.2.2. Catálogo de Dados (Unity Catalog - Schema gold_olist)	11
4.2.2.1. Tabela Fato Enriquecida: gold_vendas_detalhadas	11
4.2.2.2. Data Mart: gold_metrics_financeiras	11
4.2.2.3. Data Mart: gold_metrics_logistica	11
4.2.2.4. Data Mart: gold_metrics_desempenho_comercial	12
5. Qualidade de Dados	12
5.1. Completude	12

5.2. Consistência	12
5.3. Unicidade	12
5.4. Acurácia e Outliers	13
6. Resultados, Validação e Análise de Dados por Domínio	13
6.1. Domínio de Finanças e Meios de Pagamento (gold_metrics_financas)	13
6.2. Domínio de Logística e Experiência do Cliente (gold_metrics_logistica)	14
6.3. Domínio de Desempenho Comercial e Cobertura de Vendedores (gold_metrics_desempenho_comercial)	15
6.4. Conclusão Integrada	15
6.5. Recomendações	15
7. Autoavaliação do Desenvolvimento	16
7.1. Atingimento dos Objetivos	16
7.2. Dificuldades Encontradas	16
7.3. Trabalhos Futuros para Aprimoramento do Portfólio	16

1. Contexto de Negócios e Formulação do Problema

1.1. Contexto de Negócios

A **Olist Store** é uma startup brasileira que atua no setor de tecnologia (e-commerce) para o varejo. A empresa oferece soluções que conectam pequenos/médios lojistas (vendedores) e grandes marcas a marketplaces, facilitando a gestão de lojas físicas e online, logística e serviços financeiros.

O conjunto de dados utilizado compreende informações de aproximadamente **100 mil pedidos** realizados na Olist Store entre **2016 e 2018**. Trata-se de dados comerciais reais anonimizados disponibilizados publicamente no Kaggle.

Necessidade do Negócio: Buscaremos identificar gargalos na jornada do cliente, entender a dinâmica financeira dos pagamentos e otimizar a distribuição geográfica da malha de vendedores. Atualmente, os dados brutos estão dispersos em múltiplos arquivos, sem modelagem analítica unificada. Para resolver essa limitação, este trabalho constrói um pipeline de dados analítico de ponta a ponta na plataforma **Databricks**, utilizando a **Arquitetura Medalhão (Bronze, Silver e Gold)** e **Unity Catalog**, culminando na construção de uma modelagem dimensional (*Star Schema*).

1.2. Perguntas de Negócio

Para nortear toda a arquitetura técnica, modelagem de dados e validações do pipeline, o trabalho responderá diretamente às seguintes perguntas estratégicas, divididas por domínio de negócio:

1.2.1. Domínio de Finanças e Meios de Pagamento

Pergunta 1: Qual é o ticket médio das transações por método de pagamento (cartão de crédito, boleto, voucher, cartão de débito)?

Pergunta 2: Qual meio de pagamento concentra o maior volume financeiro total e qual apresenta o maior número médio de parcelas?

Objetivo de Análise: Mapear o perfil de pagamento dos clientes para otimizar taxas operacionais e estratégias de liquidez.

1.2.2. Domínio de Logística e Experiência do Cliente

Pergunta 1: Qual é o tempo médio de entrega (em dias corridos desde a compra até a entrega efetiva) agrupado por estado de destino do cliente?

Pergunta 2: Quais são os 5 estados com os maiores índices de atraso na entrega (percentual de pedidos entregues após a data estimada de entrega)?

Objetivo de Análise: Mapear a eficiência da malha logística nacional, identificar estados críticos e fundamentar renegociações de prazos com transportadoras regionais.

1.2.3. Domínio de Desempenho Comercial e Cobertura de Vendedores

Pergunta 1: Qual é a distribuição geográfica dos vendedores em termos de densidade e receita gerada?

Pergunta 2: Quais estados apresentam maior volume de vendas em valor e itens comercializados, e qual a participação percentual dos vendedores locais no atendimento dessas demandas?

Objetivo de Análise: Mapear a concentração da oferta de produtos e direcionar os times de expansão comercial para a prospecção de novos vendedores em regiões de alta demanda e pouca oferta local.

1.3. Especificação do Conjunto de Dados Brutos e Licenciamento

Fonte de Origem: Repositório Público Kaggle (*E-Commerce Public Dataset by Olist*).

Licença de Uso: CC BY-NC-SA 4.0 (Permite uso educacional e não comercial com a devida atribuição de autoria).

Estrutura de Arquivos Brutos: uso de 6 tabelas relacionais, discriminadas abaixo, em formato CSV conectadas por chaves primárias e estrangeiras (order_id, customer_id, product_id, seller_id):

Tabela	Função no Pipeline	Atributos Principais	Chaves de Relacionamento
olist_orders_dataset	Informações dos Pedidos (Entidade Central)	order_status order_purchase_timestamp order_delivered_customer_date order_estimated_delivery_date	order_id (PK) customer_id (FK)
olist_customers_dataset	Cadastro de Clientes	customer_zip_code_prefix customer_city customer_state	customer_id (PK)
olist_order_items_dataset	Detalhamento dos Itens do Pedido	order_item_id price freight_value shipping_limit_date	order_id (FK) product_id (FK) seller_id (FK)
olist_order_payments_dataset	Transações Financeiras	payment_sequential payment_type payment_installments payment_value	order_id (FK)
olist_products_dataset	Catálogo de Produtos	product_category_name product_weight_g product_length_cm	product_id (PK)
olist_sellers_dataset	Cadastro de Vendedores	seller_city seller_state	seller_id (PK)

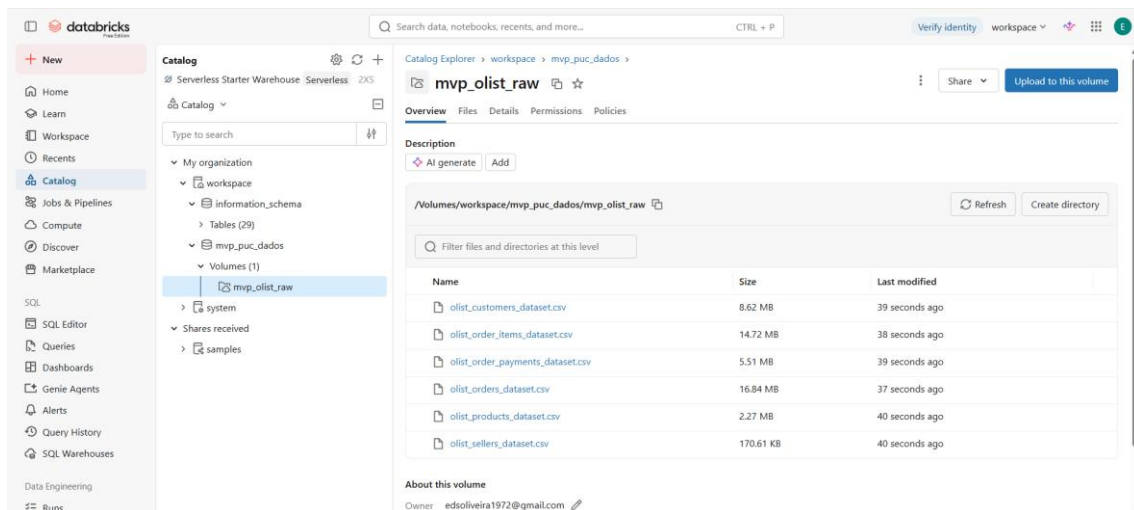
2. Ingestão e Carga dos Dados Brutos

2.1. Estratégia de Ingestão

Para simular um ambiente de nuvem corporativo e escalável, a ingestão foi projetada em lote (*batch*). Os arquivos originais foram centralizados na nuvem no ambiente Databricks Unity Catalog Volumes.

2.2. Procedimento Operacional da Carga

1. **Download Local:** Extração dos 6 arquivos CSV originais do Kaggle.
2. **Upload para a Nuvem:** Utilizou-se a interface web do Databricks: New → Add or upload data → Upload files to a volume → Seleção do Schema e Volume no Unity Catalog.
3. **Persistência Inicial:** Os arquivos brutos ficaram armazenados de forma imutável no diretório de volume para posterior leitura pela camada Bronze.



3. Arquitetura do Pipeline de Dados e Processamento Bronze

3.1. Organização do Pipeline

O pipeline ETL foi ramificado em três notebooks encadeados para assegurar modularidade, fácil manutenção e separação das camadas da Arquitetura Medalhão:

01_ingestao_bronze: Leitura dos arquivos CSV brutos e gravação no formato Delta Lake.

02_ingestao_silver: Limpeza, tratamento de tipo, padronização textual e criação das entidades limpas.

03_ingestao_gold: Construção da Tabela Fato unificada e Tabelas Agregadas para resposta direta às perguntas de negócio.

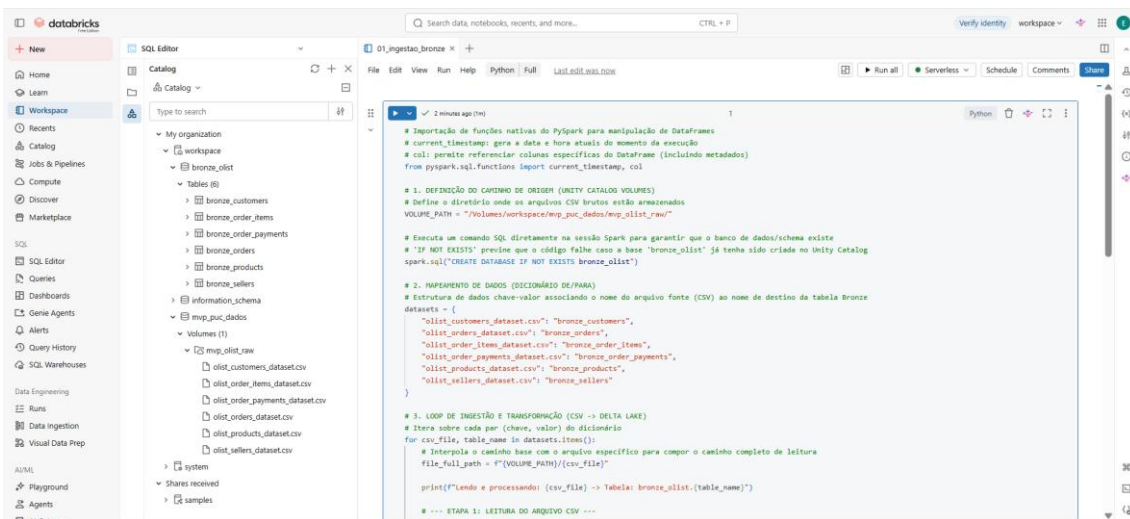
3.2. Camada Bronze (01_ingestao_bronze)

A camada Bronze armazena os dados no formato otimizado Delta Lake mantendo a granularidade original sem alterações de regras de negócio. Foram adicionados metadados para auditoria de pipeline:

_ingestion_time: Timestamp da data e hora da gravação no Data Lake (current_timestamp()).

_source_file: Caminho e nome do arquivo de origem.

O Notebook Python **01_ingestao_bronze**, anexado no Github em [https://github.com/edsoliveira72/MVP_PUC_Engenharia_Dados/blob/main/01_ingestao_bronze](https://github.com/edsoliveira72/MVP_PUC_Engenharia_Dados/blob/main/01_ingestao_bronze.ipynb) .ipynb faz um processo de ingestão de dados (arquivos CSV para tabelas em formato Delta Lake) dentro do ambiente Databricks, utilizando PySpark e o recurso de Unity Catalog.



```
# Importação de funções nativas do PySpark para manipulação de DataFrames
# current_timestamp: gera a data e hora atuais do momento da execução
# col: permite referenciar colunas específicas do Dataframe (incluindo metadados)
from pyspark.sql.functions import current_timestamp, col

# 1. DEFINIÇÃO DO CAMINHO DE ORIGEM (UNITY CATALOG VOLUMES)
# Define o diretório onde os arquivos CSV brutos estão armazenados
VOLUME_PATH = "/Volumes/workspace/mvp_puc_dados/mvp_olist_raw"

# Executa um comando SQL diretamente na sessão Spark para garantir que o banco de dados/schema existe
# 'IF NOT EXISTS' previne que o código falhe caso a base 'bronze_olist' já tenha sido criada no Unity Catalog
spark.sql("CREATE DATABASE IF NOT EXISTS bronze_olist")

# 2. Mapeamento de dados (DICIONÁRIO DE/ PARA)
# Estrutura de dados chave-valor associando o nome do arquivo fonte (CSV) ao nome de destino da tabela Bronze
datasets = {
    "olist_customers_dataset.csv": "bronze_customers",
    "olist_orders_dataset.csv": "bronze_orders",
    "olist_order_items_dataset.csv": "bronze_order_items",
    "olist_order_payments_dataset.csv": "bronze_order_payments",
    "olist_products_dataset.csv": "bronze_products",
    "olist_sellers_dataset.csv": "bronze_sellers"
}

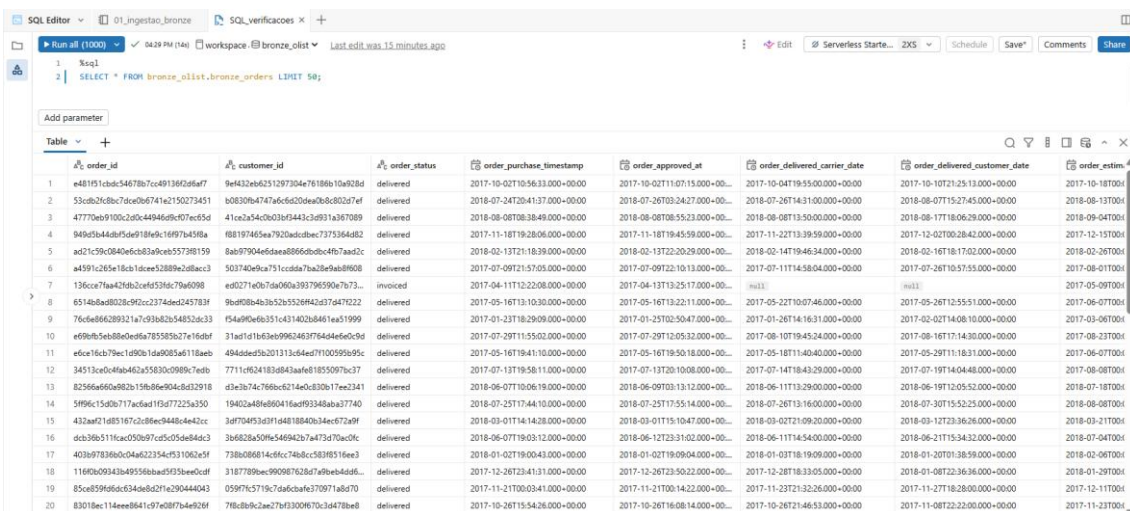
# 3. LOOP DE INGESTÃO E TRANSFORMAÇÃO (CSV -> DELTA LAKE)
# Itera sobre cada par (chave, valor) do dicionário
for csv_file, table_name in datasets.items():
    # Interpola o caminho base com o arquivo específico para compor o caminho completo de leitura
    file_full_path = f"{VOLUME_PATH}/{csv_file}"

    print(f" lendo e processando: {csv_file} -> Tabela: bronze_olist.{table_name}")

    --- ETAPA 1: LECTURA DO ARQUIVO CSV ---
```

Código SQL para inspecionar uma das tabelas criadas:

SELECT * FROM bronze_olist.bronze_orders LIMIT 50;



order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estim.
e481651cd454678b7cc491362d6a7	9e432e6251297304e76188b10a628d	delivered	2017-10-02T10:56:33.000+00:00	2017-10-02T11:07:15.000+00:00	2017-10-04T19:55:00.000+00:00	2017-10-10T21:25:13.000+00:00	2017-10-18T00:00:00.000+00:00
53cb26c8b7c4e0b6741e2150273451	b082094747e6c6c02de4b0e802b7e7	delivered	2018-07-24T20:41:37.000+00:00	2018-07-26T02:24:27.000+00:00	2018-07-26T14:31:00.000+00:00	2018-08-07T15:27:45.000+00:00	2018-08-13T00:00:00.000+00:00
47770e9f100c3a0c44946d9c407ec5d	41ca2a54c0b30359443c3a931a367089	delivered	2018-08-08T08:38:49.000+00:00	2018-08-08T08:55:23.000+00:00	2018-08-08T13:50:00.000+00:00	2018-08-17T18:06:29.000+00:00	2018-09-04T00:00:00.000+00:00
949e5b44b5d9f18e6a16e9b1697b458a	888197465ea7920ad0bce7375364682	delivered	2017-11-18T19:28:06.000+00:00	2017-11-18T19:45:59.000+00:00	2017-11-22T13:39:59.000+00:00	2017-12-02T08:28:42.000+00:00	2017-12-15T00:00:00.000+00:00
ad21c59c840e6c83a9ce55738159	8ab97904e5a8a886d0bc4b7aa2dc	delivered	2018-02-13T21:18:39.000+00:00	2018-02-13T22:29:29.000+00:00	2018-02-14T19:46:34.000+00:00	2018-02-16T18:17:02.000+00:00	2018-02-26T00:00:00.000+00:00
a4591c265e18c1dce52889e2d8ac3	503740e6ca751ccdda7aa38e9ab8f608	delivered	2017-07-09T21:57:05.000+00:00	2017-07-09T22:10:13.000+00:00	2017-07-11T14:58:04.000+00:00	2017-07-26T10:57:55.000+00:00	2017-08-01T00:00:00.000+00:00
136cca7aa42fb2c8d3f6c79a6098	ed0271e07c1da006a393796590e7b73	invoiced	2017-04-11T12:22:08.000+00:00	2017-04-13T13:25:17.000+00:00	null	null	2017-05-09T00:00:00.000+00:00
6514ba8d028c9d2c374ded245783f	9bd0f0e4b3b52525642d37d470222	delivered	2017-05-16T13:10:30.000+00:00	2017-05-16T13:22:11.000+00:00	2017-05-22T10:07:46.000+00:00	2017-05-26T12:55:51.000+00:00	2017-06-07T00:00:00.000+00:00
76de6a6d289321a7c93b2b54852dc33	f54a0f0e6351c431402b28461ea51999	delivered	2017-01-23T18:29:09.000+00:00	2017-01-25T02:50:47.000+00:00	2017-01-26T14:16:31.000+00:00	2017-02-02T14:08:10.000+00:00	2017-03-08T00:00:00.000+00:00
ed9eb5eb88e0e6a785585b27e16dbf	31ad1d1b63e999634637644a4e6c0d4	delivered	2017-07-29T11:55:02.000+00:00	2017-07-29T12:05:32.000+00:00	2017-08-10T19:45:24.000+00:00	2017-08-16T17:14:30.000+00:00	2017-08-23T00:00:00.000+00:00
ef0ec16cb79ec1d90b1da0808a6118ab	494d4ed5b201313d4ad7110059595c	delivered	2017-05-16T19:41:10.000+00:00	2017-05-16T19:50:18.000+00:00	2017-05-18T11:40:40.000+00:00	2017-05-29T11:18:31.000+00:00	2017-06-07T00:00:00.000+00:00
34513ced0c4ab462a55830c0896c7ebd	7711cd624183d843aaf6d1855097bc37	delivered	2017-07-13T19:58:11.000+00:00	2017-07-13T20:10:08.000+00:00	2017-07-14T18:43:29.000+00:00	2017-07-19T14:04:48.000+00:00	2017-08-08T00:00:00.000+00:00
82566a660e802b15b68e904c8d32918	d3e3b74c786bc6214a0c30b17e2c241	delivered	2018-06-07T10:06:19.000+00:00	2018-06-09T03:13:12.000+00:00	2018-06-11T13:29:00.000+00:00	2018-06-19T12:05:52.000+00:00	2018-07-18T00:00:00.000+00:00
5996c15d0b7174adad11d77225a350	19402a48e680416ad993348ab37740	delivered	2018-07-25T17:44:10.000+00:00	2018-07-25T17:55:14.000+00:00	2018-07-26T13:16:00.000+00:00	2018-07-30T15:52:25.000+00:00	2018-08-08T00:00:00.000+00:00
432aaf21d85167c2c88e9444c4e42cc	3df704953d3f1d4818840b34ac672a9f	delivered	2018-03-01T14:14:28.000+00:00	2018-03-01T15:10:47.000+00:00	2018-03-02T21:09:20.000+00:00	2018-03-12T23:36:26.000+00:00	2018-03-21T00:00:00.000+00:00
dc3b9b511cac050697cd5c05de84dc3	36a828a509a546942b7a473d70a0c0c	delivered	2018-06-07T19:03:12.000+00:00	2018-06-12T23:31:02.000+00:00	2018-06-11T14:54:00.000+00:00	2018-06-21T15:34:32.000+00:00	2018-07-04T00:00:00.000+00:00
403b978360c0462234c4f313062e5f	738b0861a46fc748dc5838516ee3	delivered	2018-01-02T19:00:43.000+00:00	2018-01-02T19:09:04.000+00:00	2018-01-03T18:19:09.000+00:00	2018-01-20T01:38:59.000+00:00	2018-02-06T00:00:00.000+00:00
116d009343b49556baad5f55ewc0df	3187789ec99086763d7a7e4eb4d4d6	delivered	2017-12-26T23:41:01.000+00:00	2017-12-26T23:50:22.000+00:00	2017-12-28T18:33:05.000+00:00	2018-01-08T22:36:36.000+00:00	2018-01-29T00:00:00.000+00:00
85c059f6dc634a8d27e29040403	05976c7519c7d6dcba6c710971a8a70	delivered	2017-11-21T00:04:10.000+00:00	2017-11-21T00:14:22.000+00:00	2017-11-23T21:32:26.000+00:00	2017-11-27T18:28:00.000+00:00	2017-12-11T00:00:00.000+00:00
83018ec114ee8641c97a087b4a926f	78b8b9c2ae27b3300670c3d478ba8	delivered	2017-10-26T19:54:26.000+00:00	2017-10-26T16:08:14.000+00:00	2017-10-26T21:46:53.000+00:00	2017-11-08T22:22:00.000+00:00	2017-11-23T00:00:00.000+00:00

4. Processamento da Camada Silver e Modelagem

4.1. Transformações e Qualidade na Camada Silver (02_ingestao_silver)

Na camada Silver, pegamos os dados brutos salvos no formato Delta da camada Bronze e aplicamos regras de limpeza, padronização e estruturação. O objetivo é deixar os dados prontos para análise e modelagem de negócios na camada Gold, mantendo a granularidade das entidades originais. O processamento da camada Silver foi realizada por meio do Notebook Python **02_ingestao_silver**, anexado no Github em https://github.com/edsoliveira72/MVP_PUC_Engenharia_Dados/blob/main/02_ingestao_silver.ipynb.

As ações executadas visam garantir a conformidade dos dados com os requisitos de qualidade, resolução de valores nulos, ajuste de tipagem e padronização textual:

a) Garantia de Contrato de Dados e Tipagem:

- Métricas de Tempo (Timestamps): Convertem-se todas as colunas de datas da tabela de pedidos (order_purchase_timestamp, order_approved_at, order_delivered_customer_date, etc.) para o tipo nativo **TIMESTAMP**.
- Valores Financeiros e Numéricos: As colunas financeiras (como price, freight_value e payment_value) foram convertidas para **DOUBLE**. Identificadores sequenciais e quantitativos (payment_installments, order_item_id, dimensões do produto) foram tipados explicitamente como **INTEGER**.
- Identificadores Geográficos: Mantiveram-se os campos de CEP (zip_code_prefix) explicitamente tipados como **STRING** em vez de numéricos, prevenindo a perda acidental de zeros à esquerda.

b) Padronização Textual:

Para evitar inconsistências em futuras consultas e agrupamentos (onde "São Paulo" e "sao paulo" seriam tratados como entradas distintas), aplicou-se:

- trim() para remoção de espaços em branco antes e depois das cadeias de caracteres.
- lower() em nomes de cidades e categorias para uniformizar registros em caixa baixa.
- upper() para as siglas de estados (customer_state e seller_state), garantindo a padronização de unidades federativas.

c) Tratamento de Nulos e Enriquecimento (De-Para):

- Na tabela de produtos (silver_products), aplicou-se a função coalesce() para preencher dados ausentes de categorias com valores padrão ("nao_definido"), garantindo que agregações posteriores não ocultem dados por conta de valores **NULL**.

d) Governança e Rastreabilidade:

- Adicionou-se a coluna de controle _processed_at (current_timestamp()) em todas as tabelas criadas. Esse metadado permite auditar exatamente quando cada registro foi processado e inserido na Camada Silver.

Organização no Unity Catalog: As tabelas resultantes foram consolidadas sob o schema dedicado `silver_olist` (`silver_orders`, `silver_order_items`, `silver_customers`, `silver_sellers`, `silver_order_payments` e `silver_products`).

```

# DLIST - CANADA SILVER
# Descrição: leitura das tabelas brutas da camada Bronze, conversão de tipos de dados,
# tratamento de nulos, padronização de textos e enriquecimento com de-paras.

from pyspark.sql.functions import col, trim, lower, upper, coalesce, lit, current_timestamp

# 1. CRIAÇÃO DO DATABASE/SCHEMA SILVER
# Garante que o schema de destino esteja criado no Unity Catalog
spark.sql("CREATE DATABASE IF NOT EXISTS silver_olist")
print("Database 'silver_olist' pronto para receber as tabelas.")

# 2. TABELA: SILVER_ORDERS (Pedidos)
# Objetivo: Converter colunas de string para TIMESTAMP para permitir cálculos de frete/strato.
print("Processando: silver_orders...")

df_orders = spark.table("bronze_olist.bronze_orders")

df_orders_silver = df_orders.select(
    col("order_id").alias("order_id"),
    col("customer_id").alias("customer_id"),
    col("order_status").alias("order_status"),

    # Conversão de strings de data para tipo TIMESTAMP oficial
    col("order_purchase_timestamp").cast("timestamp").alias("order_purchase_timestamp"),
    col("order_approved_at").cast("timestamp").alias("order_approved_at"),
    col("order_delivered_carrier_date").cast("timestamp").alias("order_delivered_carrier_date"),
    col("order_delivered_customer_date").cast("timestamp").alias("order_delivered_customer_date"),
    col("order_estimated_delivery_date").cast("timestamp").alias("order_estimated_delivery_date"),

    # Metadado de processamento da camada Silver
    current_timestamp().alias("_processed_at")
)
  
```

Código SQL para inspecionar uma das tabelas criadas:

SELECT * FROM silver_olist.silver_orders LIMIT 50;

	order_id	customer_id	order_status	order_purchase_timestamp	order_approved_at	order_delivered_carrier_date	order_delivered_customer_date	order_estimated_delivery_date
1	448151c8dc54678a7c049138f084f7	9e4324e4251297304a761886b10a03d0	delivered	2017-10-02T11:07:15.000-00:00	2017-10-04T19:55:00.000-00:00	2017-10-10T21:25:13.000-00:00	2017-10-18T05:00:00.000-00:00	2017-10-18T05:00:00.000-00:00
2	53c0b26f8c1e0c0d741e2150273451	b0830b47474dc6d020a0b0d02a7af	delivered	2018-07-24T20:41:37.000-00:00	2018-07-28T03:24:27.000-00:00	2018-07-28T14:31:00.000-00:00	2018-08-07T15:27:45.000-00:00	2018-08-13T00:00:00.000-00:00
3	47770e0100c2a0c44846d9f07ac5d4	41e2a54c03034431c3891a3670f89	delivered	2018-08-08T08:38:49.000-00:00	2018-08-08T08:38:49.000-00:00	2018-08-08T13:30:00.000-00:00	2018-08-17T18:06:28.000-00:00	2018-08-04T00:00:00.000-00:00
4	94845d44d8f5d8f18f6c168f7b458a	688197485ea7920edc0bc7275364682	delivered	2017-11-18T19:28:06.000-00:00	2018-02-13T22:20:29.000-00:00	2017-11-22T13:38:59.000-00:00	2017-12-02T00:28:42.000-00:00	2017-12-13T05:00:00.000-00:00
5	a021c150840f6c3d3dc4e55738f159	8a07904d64a0a056d8d0c4b7a0d2c	delivered	2018-02-13T21:18:39.000-00:00	2018-02-13T22:20:29.000-00:00	2018-02-14T19:46:34.000-00:00	2018-02-16T18:17:02.000-00:00	2018-02-26T00:00:00.000-00:00
6	a4591c265e1f0c1d0e52889c2d8acc1	503740e0a751c0dda7ba28eab8908	delivered	2017-07-09T22:10:13.000-00:00	2017-07-09T22:10:13.000-00:00	2017-07-11T14:58:04.000-00:00	2017-07-26T19:57:55.000-00:00	2017-08-01T00:00:00.000-00:00
7	136fce7a4a03b0b3d9f336c78a098	e05271a076da006a39379890a76731	delivered	2017-04-11T11:22:08.000-00:00	2017-04-11T11:22:08.000-00:00	2017-04-13T13:25:17.000-00:00	2017-05-09T00:00:00.000-00:00	2017-05-09T00:00:00.000-00:00
8	69148a0028c9f0c2374d4e04578f9	9e0f9b4b3652652594a2d70470232	delivered	2017-05-18T15:10:30.000-00:00	2017-05-18T15:10:30.000-00:00	2017-05-22T10:07:48.000-00:00	2017-05-26T12:55:1.000-00:00	2017-06-07T00:00:00.000-00:00
9	76c4d00289321e094062a14835b13	04a9f94b0315a114203a04f0e11989	delivered	2017-01-23T02:56:47.000-00:00	2017-01-23T02:56:47.000-00:00	2017-01-28T14:18:31.000-00:00	2017-02-07T14:08:10.000-00:00	2017-02-07T00:00:00.000-00:00
10	49f84d5081b0e0e67878970c27c16d6f	31ae1af19f3a099630439764d6a6c04	delivered	2017-07-28T10:03:32.000-00:00	2017-07-28T10:03:32.000-00:00	2017-08-16T11:14:30.000-00:00	2017-08-16T11:14:30.000-00:00	2017-08-23T00:00:00.000-00:00
11	ef0a13c0f49e4402a15030c089b7e1b	44d4a0d201313d4aef7102959a05c	delivered	2017-05-18T19:58:11.000-00:00	2017-05-18T19:58:11.000-00:00	2017-05-18T19:58:11.000-00:00	2017-05-26T11:18:31.000-00:00	2017-06-07T00:00:00.000-00:00
12	34513c0f49e4402a15030c089b7e1b	77114d2d18348b3a4e6185509a7c17	delivered	2017-07-13T19:58:11.000-00:00	2017-07-13T19:58:11.000-00:00	2017-07-14T18:43:29.000-00:00	2017-07-19T14:04:48.000-00:00	2017-08-08T00:00:00.000-00:00
13	8256a6a0a062a15030c089b7e1b	d3a3c74c786b0214a0c320b17ee2341	delivered	2018-06-07T10:06:19.000-00:00	2018-06-08T03:13:12.000-00:00	2018-06-11T13:26:00.000-00:00	2018-06-19T12:55:25.000-00:00	2018-07-18T00:00:00.000-00:00
14	5906c15d0b717ac6ad193477225350	19403a480a00416a0f93348a037740	delivered	2018-07-25T17:53:14.000-00:00	2018-07-25T17:53:14.000-00:00	2018-07-30T15:52:25.000-00:00	2018-08-08T00:00:00.000-00:00	2018-08-08T00:00:00.000-00:00
15	432aa2f185167c2c86e9448c4e42c	3a070453f164818840b344c672a9f	delivered	2018-03-01T14:14:28.000-00:00	2018-03-01T15:10:47.000-00:00	2018-03-02T12:09:20.000-00:00	2018-03-12T23:36:26.000-00:00	2018-03-21T00:00:00.000-00:00
16	dcb36b511fa0509f7c5c5de8a4c3	3a6828a509c546942b747370c0e3	delivered	2018-06-07T19:03:12.000-00:00	2018-06-07T19:03:12.000-00:00	2018-06-11T14:54:00.000-00:00	2018-06-21T15:34:32.000-00:00	2018-07-04T00:00:00.000-00:00
17	403b67836b0c4a622354c53106d5f	738b086814d6c74b8c5e838516e03	delivered	2018-01-02T19:09:04.000-00:00	2018-01-02T19:09:04.000-00:00	2018-01-03T18:19:09.000-00:00	2018-01-20T01:38:59.000-00:00	2018-02-06T00:00:00.000-00:00
18	1168b0934304859586a0593b0e0c0f	31877898c990876236f49b0e4a6d6	delivered	2017-12-28T23:50:22.000-00:00	2017-12-28T23:50:22.000-00:00	2018-01-08T22:36:36.000-00:00	2018-01-08T22:36:36.000-00:00	2018-01-29T00:00:00.000-00:00
19	85cae596d6a34a0e221e2004404d3	05967c5719c7d6acaf6370971ab70	delivered	2017-11-21T00:03:41.000-00:00	2017-11-21T00:03:41.000-00:00	2017-11-23T21:32:26.000-00:00	2017-11-27T18:28:00.000-00:00	2017-12-11T00:00:00.000-00:00
20	83018ec114ee8641c97a08704e03f	78b0b9c2ae270f3300670c3d478b0d	delivered	2017-10-28T16:08:14.000-00:00	2017-10-28T16:08:14.000-00:00	2017-10-28T16:08:14.000-00:00	2017-11-27T18:28:00.000-00:00	2017-12-11T00:00:00.000-00:00
21								

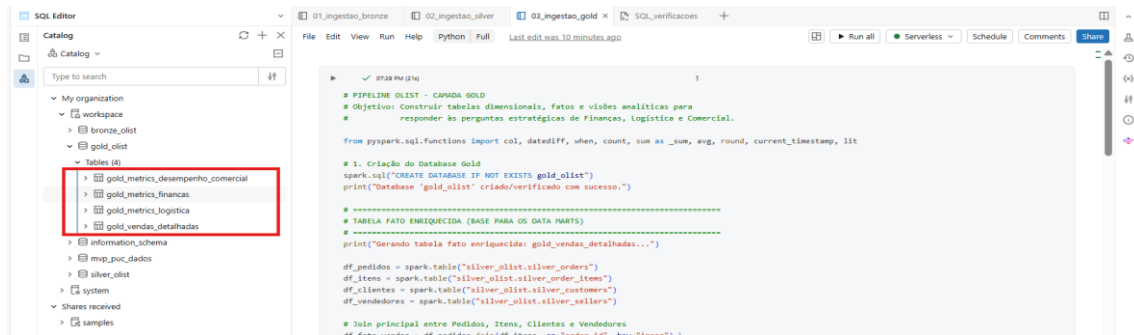
4.2. Modelagem Dimensional (Star Schema) e Catálogo de Dados

O modelo analítico foi desenvolvido seguindo o padrão **Star Schema (Esquema Estrela)** para otimizar o desempenho de consultas na camada Gold e alimentar os *Data Marts* operacionais.

A arquitetura orientada ao código PySpark é estruturada a partir da tabela fato central enriquecida **gold_vendas_detalhadas** (que realiza o *join* das entidades da camada Silver), servindo de base para o processamento dos *Data Marts* analíticos específicos de cada domínio.

Nesta camada, a Tabela Fato Agregada (**gold_vendas_detalhadas**) e tabelas agregadas específicas para cada domínio de negócio foram criadas, por meio do Notebook Python **03_ingestao_gold**, anexado no Github em https://github.com/edsoliveira72/MVP_PUC_Engenharia_Dados/blob/main/03_ingestao_gold.ipynb.

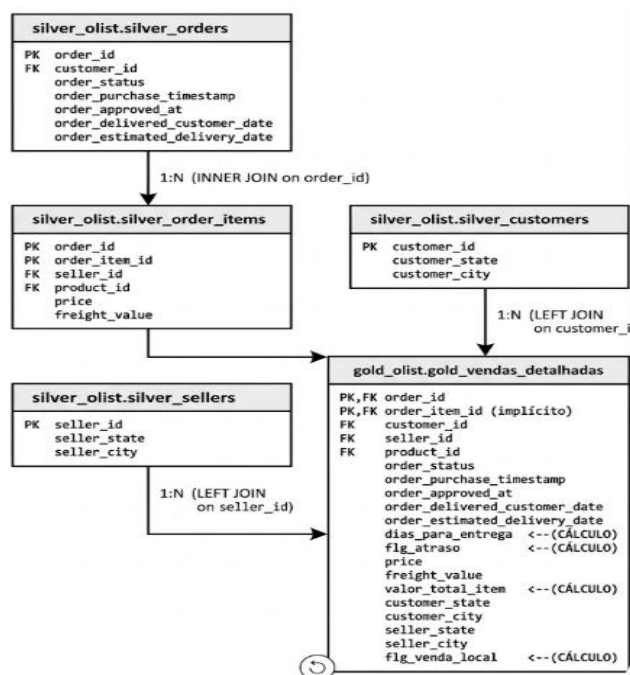
- **gold_metrics_financas;**
- **gold_metrics_logistica;**
- **gold_metrics_desempenho_comercial.**



Na primeira célula do notebook 03_ingestao_gold consolidamos o relacionamento dos pedidos com seus itens, clientes, vendedores e pagamentos.

Na segunda célula do notebook 03_ingestao_gold criamos 3 tabelas de suporte para responder diretamente a cada um dos 3 domínios estabelecidos nas perguntas de negócio (**Finanças e Meios de Pagamento, Logística e Experiência do Cliente e Desempenho Comercial e Cobertura de Vendedores**).

4.2.1. Esquema Estrela Tabela gold_vendas_detalhadas



▼ gold_vendas_detalhadas

order_id
customer_id
seller_id
product_id
order_status
order_purchase_timestamp
order_approved_at
order_delivered_customer_date
order_estimated_delivery_date
dias_para_entrega
flg_atraso
price
freight_value
valor_total_item
customer_state
customer_city
seller_state
seller_city
flg_venda_local

4.2.2. Catálogo de Dados (Unity Catalog - Schema gold_olist)

4.2.2.1. Tabela Fato Enriquecida: gold_vendas_detalhadas

Contexto: Tabela fato consolidada por item de pedido, contendo métricas calculadas de SLA (Service Level Agreement - Acordo de Nível de Serviço) de entrega e classificação de venda local.

Junção (inner/left) de silver_olist.silver_orders, silver_olist.silver_order_items, silver_olist.silver_customers e silver_olist.silver_sellers.

Campo	Tipo de Dado	Descrição e Regras de Negócio	Domínio / Valores Esperados
order_id	STRING	Identificador único do pedido	Hash UUID de 32 caracteres
customer_id	STRING	Chave de ligação com o cliente	Hash UUID de 32 caracteres
seller_id	STRING	Chave de ligação com o vendedor	Hash UUID de 32 caracteres
product_id	STRING	Chave de ligação com o produto	Hash UUID de 32 caracteres
order_status	STRING	Status atual do processamento do pedido	delivered, shipped, canceled, invoiced, etc.
order_purchase_timestamp	TIMESTAMP	Data e hora em que a compra foi realizada	Formato YYYY-MM-DD HH:MM:SS
order_approved_at	TIMESTAMP	Data e hora de aprovação do pagamento	Formato YYYY-MM-DD HH:MM:SS
order_delivered_customer_date	TIMESTAMP	Data e hora efetiva da entrega ao cliente	Formato YYYY-MM-DD HH:MM:SS (Pode conter nulos)
order_estimated_delivery_date	TIMESTAMP	Data estimada de entrega informada no checkout	Formato YYYY-MM-DD HH:MM:SS
dias_para_entrega	INTEGER	Cálculo do tempo decorrido: datediff(order_delivered_customer_date, order_purchase_timestamp)	Inteiros > = 0 (em dias corridos)
flg_atraso	INTEGER	Flag indicativo de atraso: when(order_delivered_customer_date > order_estimated_delivery_date, 1) otherwise(0)	0 (Em dia / Antecipado), 1 (Com atraso)
price	DOUBLE	Preço unitário do item comercializado	Mín: R\$ 0,85, Máx: R\$ 6.729,00
freight_value	DOUBLE	Valor do frete atribuído ao item	Mín: R\$ 0,00, Máx: R\$ 409,68
valor_total_item	DOUBLE	Soma do preço e frete: (price + freight_value)	Valores numéricos decimais em Reais (R\$)
customer_state	STRING	Sigla do estado do cliente comprador	27 UFs do Brasil (SP, RJ, MG, etc.)
customer_city	STRING	Nome padronizado da cidade do cliente	Nomes de municípios em caixa baixa
seller_state	STRING	Sigla do estado do vendedor	27 UFs do Brasil (SP, PR, SC, etc.)
seller_city	STRING	Nome padronizado da cidade do vendedor	Nomes de municípios em caixa baixa
flg_venda_local	INTEGER	Flag de atendimento regional: when(customer_state == seller_state, 1) otherwise(0)	0 (Venda interestadual), 1 (Venda intraestadual)

4.2.2.2. Data Mart: gold_metrics_financeiras

Contexto: Tabela analítica agregada por modalidade de pagamento para avaliação de desempenho financeiro e perfil de parcelamento.

Agregação da tabela silver_olist.silver_order_payments.

Campo	Tipo de Dado	Descrição e Regras de Negócio	Domínio / Valores Esperados
payment_type	STRING	Método de pagamento utilizado no pedido	credit_card, boleto, voucher, debit_card
ticket_medio_pagamento	DOUBLE	Média do valor transacionado: round(avg(payment_value), 2)	Valores numéricos decimais em Reais (R\$)
receita_total_pagamento	DOUBLE	Soma do volume transacionado: round(sum(payment_value), 2)	Valores numéricos decimais em Reais (R\$)
total_transacoes	LONG	Volume total de transações processadas: count(order_id)	Números inteiros >= 0
media_parcelas	DOUBLE	Média de parcelas escolhidas: round(avg(payment_installments), 2)	Mín: 1, Máx: 24

4.2.2.3. Data Mart: gold_metrics_logistica

Contexto: Tabela analítica de acompanhamento do nível de serviço (SLA) de entregas agrupada por estado do cliente.

Agregação da tabela gold_olist.gold_vendas_detalhadas filtrando apenas order_status == 'delivered'.

Campo	Tipo de Dado	Descrição e Regras de Negócio	Domínio / Valores Esperados
customer_state	STRING	Estado de destino do comprador (UF)	27 UFs brasileiras
tempo_medio_entrega_dias	DOUBLE	Média em dias para a entrega: round(avg(dias_para_entrega), 2)	Valores decimais em dias corridos
total_pedidos_entregues	LONG	Contagem de pedidos com status entregue: count(order_id)	Números inteiros >= 0
total_pedidos_atrasados	LONG	Quantidade de pedidos entregues após a data estimada: sum(flz_atraso)	Números inteiros >= 0
pct_pedidos_atrasados	DOUBLE	Percentual de inadimplência no SLA: round((sum(flz_atraso) / count(order_id)) * 100, 2)	Percentual entre 0,00% e 100,00%

4.2.2.4. Data Mart: gold_metrics_desempenho_comercial

Contexto: Tabela analítica de volumetria comercial e medição da cobertura e dependência da malha de vendedores por estado.

Agregação da tabela gold_olist.gold_vendas_detalhadas agrupando por customer_state.

Campo	Tipo de Dado	Descrição e Regras de Negócio	Domínio / Valores Esperados
customer_state	STRING	Estado consumidor analisado (UF)	27 UFs brasileiras
total_itens_comercializados	LONG	Contagem total de itens vendidos para o estado: count(order_id)	Números inteiros >= 0
receita_total_estado	DOUBLE	Soma do valor total dos itens (preço + frete): round(sum(valor_total_item), 2)	Valores numéricos decimais em Reais (R\$)
qtd_itens_vendedor_local	LONG	Volume de itens vendidos por vendedores da mesma UF: sum(flag_venda_local)	Números inteiros >= 0
pct_participacao_vendedor_local	DOUBLE	Taxa de autossuficiência comercial do estado: round((sum(flag_venda_local) / count(order_id)) * 100, 2)	Percentual entre 0,00% e 100,00%

5. Qualidade de Dados

A avaliação da qualidade de dados no pipeline abordou as 5 dimensões essenciais para garantir dados confiáveis no ambiente produtivo:



5.1. Completeness:

Problema Identificado: Na tabela bruta de produtos (olist_products_dataset), 610 registros apresentavam valor nulo (NULL) para a categoria do produto.

Solução Aplicada: Aplicação da função coalesce(product_category_name, 'nao_definido') na camada Silver, impedindo a perda de dados em agrupamentos analíticos.

5.2. Consistency:

Problema Identificado: Inconsistência de maiúsculas/minúsculas e espaços em branco em nomes de cidades e estados (ex: "Sao Paulo ", "sp").

Solução Aplicada: Limpeza textual padronizada com trim(), lower() para cidades/categorias e upper() para siglas de estados.

5.3. Uniqueness:

Problema Identificado: Risco de duplicação de pagamentos e registros de transação.

Solução Aplicada: Validação de unicidade das chaves primárias (order_id, product_id, customer_id, seller_id) antes das junções relacionais para criação da Tabela Fato Gold.

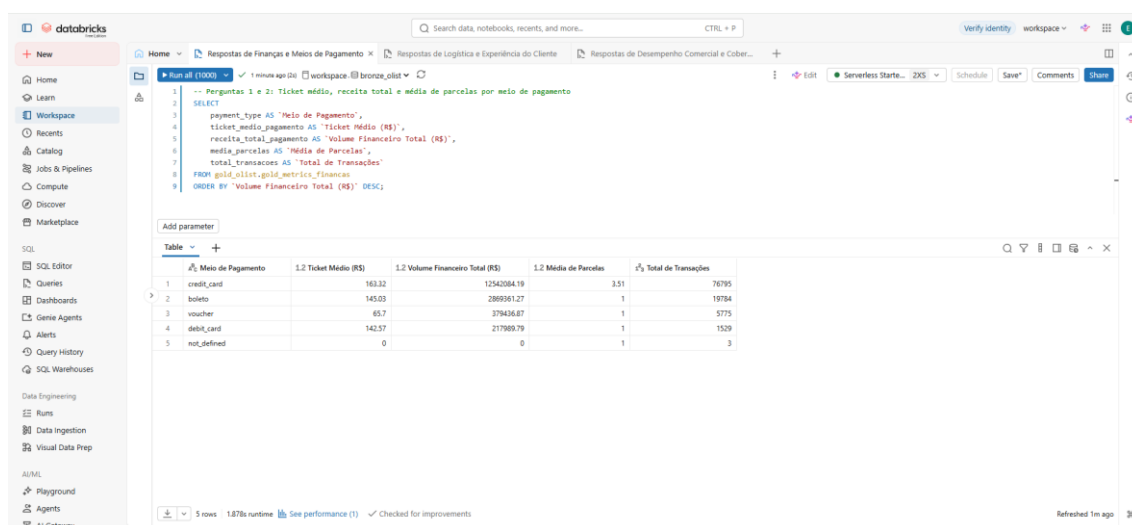
5.4. Acurácia e Outliers:

Problema Identificado: Prazos de entrega negativos resultantes de inconsistências no registro de sistema e entregas com discrepâncias extremas de datas (>200 dias).

Solução Aplicada: Filtragem explícita na camada Gold desconsiderando entregas com tempo_entrega_dias < 0. Os prazos extremamente longos para regiões remotas foram mantidos por representarem a realidade operacional do transporte rodoviário nacional.

6. Resultados, Validação e Análise de Dados por Domínio

6.1. Domínio de Finanças e Meios de Pagamento - Tabela de Resultados (gold_metrics_financas)



The screenshot shows the Databricks SQL interface. The query editor at the top contains a SQL query that filters for payment methods and calculates various metrics. Below the query, a table displays the results. The table has five columns: 'Meio de Pagamento', '1.2 Ticket Médio (R\$)', '1.2 Volume Financeiro Total (R\$)', '1.2 Média de Parcelas', and '1.2 Total de Transações'. The results are ordered by 'Volume Financeiro Total (R\$)' in descending order.

Meio de Pagamento	1.2 Ticket Médio (R\$)	1.2 Volume Financeiro Total (R\$)	1.2 Média de Parcelas	1.2 Total de Transações
credit_card	163,32	12542084,19	3,51	76795
boleto	145,03	2889361,27	1	19784
voucher	65,7	379436,67	1	5775
debit_card	142,57	217989,79	1	1529
not_defined	0	0	1	3

Análise do resultado:

Resposta 1 (Ticket Médio): O cartão de crédito apresenta o maior ticket médio (R\$ 163,14), superando o boleto (R\$ 145,03) e o cartão de débito (R\$ 142,57). O voucher registra o menor ticket médio (R\$ 65,70) por funcionar essencialmente como vale-compras complementar.

Resposta 2 (Volume e Parcelamento): O cartão de crédito concentra 78,3% do volume financeiro total (mais de R\$ 12,5 milhões) e é o único meio com parcelamento ativo (média de 3,51 parcelas). O boleto bancário ocupa o 2º lugar (~18% do faturamento).

Diagnóstico de Negócio: O crédito parcelado é o principal habilitador de conversão do e-commerce. No entanto, a dependência excessiva do cartão gera custos relevantes de taxas de adquirente e antecipação de recebíveis para os lojistas.

6.2. Domínio de Logística e Experiência do Cliente - Tabela dos 5 Estados com Maior Atraso na Entrega (gold_metrics_logistica)

Home

Learn

Workspace

Recent

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Run all (1000)

workspace: default

Last edit was now

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

Table

1.2 Estado de Destino

1.2 Tempo Médio de Entrega (dias)

1.2 Índice de Atraso (%)

1.2 total_pedidos_entregues

1.2 total_pedidos_atrasados

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

6.3. Domínio de Desempenho Comercial e Cobertura de Vendedores - Tabela de Atendimento Comercial por Estado (gold_metrics_desempenho_comercial)

SQL Editor

```
-- Pergunta 1 e 2: Vendas por estado do cliente e % de participação de vendedores locais
SELECT
  customer_state AS "Estado Consumidor",
  receita_total_estado AS "Receita Total Consumida (R$)",
  total_itens_comercializados AS "Total Itens Adquiridos",
  pct_participacao_vendedor_local AS "Venda Local (% Atendida na UF)"
FROM gold_metrics_desempenho_comercial
ORDER BY receita_total_estado DESC;
```

	1 Estado Consumidor	2 Receita Total Consumida (R\$)	3 Total Itens Adquiridos	4 Venda Local (% Atendida na UF)
1	SP	5921678.12	47449	76.28
2	RJ	2129681.98	14379	7.7
3	MG	1856161.49	13129	13.02
4	RS	885826.76	6235	5.24
5	PR	800935.44	5740	14.41
6	BA	611506.67	3799	2.05
7	SC	490215.6	4376	7.45
8	DF	393229.44	2406	2.54
9	GO	347706.92	2322	1.67
10	ES	324801.91	2256	0.4
11	PE	322237.69	1806	1.33
12	CE	275606.3	1478	0.61
13	PA	217647.11	1080	0
14	MT	186168.96	1055	0.66
15	MA	151171.99	824	1.82

Análise do resultado:

Resposta 1 (Distribuição de Vendedores): Há uma altíssima concentração de oferta no Eixo Sul-Sudeste: mais de 70% da base ativa de vendedores está sediada no estado de São Paulo.

Resposta 2 (Volume e Venda Local): São Paulo é o maior mercado consumidor (R\$ 5,92 mi) e consegue suprir 76,28% de sua própria demanda com vendedores locais. Em contrapartida, estados com alto volume de compras como Rio de Janeiro (R\$ 2,12 mi) e Minas Gerais (R\$ 1,85 mi) dependem de vendedores externos para suprir 92,3% e 86,98% de suas demandas, respectivamente. No Nordeste, a taxa de atendimento local é inferior a 3%.

Diagnóstico de Negócio: A concentração da oferta em São Paulo é a causa raiz direta das falhas logísticas identificadas no Domínio 2. Como quase todas as mercadorias saem de SP com destino a outros estados, as distâncias percorridas encarecem o frete e multiplicam os prazos de transporte.

6.4. Conclusão Integrada

O desequilíbrio na distribuição de vendedores obriga mercadorias a atravessarem o país a partir de São Paulo para abastecer praças consumidoras relevantes como RJ, MG e a Região Nordeste.

Os custos elevados de frete encarecem o valor final da compra, forçando o cliente a utilizar o parcelamento no cartão de crédito em até 3,5 vezes para diluir os custos operacionais do pedido.

6.5. Recomendações

a) Expansão Comercial Direcionada: Prospectar e integrar vendedores locais nos estados do Rio de Janeiro, Minas Gerais e em hubs estratégicos do Nordeste (Ceará/Pernambuco).

b) Rede de Operações Regionais: Implementar centros de distribuição descentralizados no Nordeste para antecipar estoques de alta rotatividade.

c) Estímulo ao PIX/Débito: Conceder descontos progressivos (ex: 3% a 5%) para compras via meios à vista, reduzindo o custo de antecipação e aumentando a margem da plataforma.

7. Autoavaliação do Desenvolvimento

7.1. Atingimento dos Objetivos

Os objetivos propostos para o MVP foram atingidos. O pipeline construído no Databricks processou com êxito os dados brutos da Olist Store, aplicando a Arquitetura Medalhão com Unity Catalog. A Tabela Fato e as Tabelas Agregadas criadas permitiram responder às 6 perguntas estratégicas com base em dados consolidados.

7.2. Dificuldades Encontradas

Aprendizado da biblioteca Pyspark, apesar de ser parecido com a biblioteca Pandas que eu já utilizo em meu trabalho.

Uso do Databricks, pois tive que aprender do zero. Contudo, considero que valeu muito a pena aprender o uso dessa ferramenta.

7.3. Trabalhos Futuros para Aprimoramento do Portfólio

a) Orquestração Automática: Implementar fluxos automatizados de agendamento e dependências via Databricks Workflows.

b) Visualização Interativa: Conectar a Camada Gold a uma ferramenta de BI (Power BI ou Databricks Dashboards) para disponibilização de painéis interativos à liderança executiva.